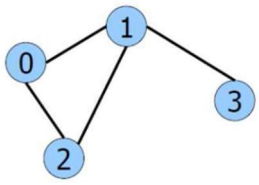
Esercitazione di laboratorio n. 4

Esercizio n.1: Vertex Cover
Competenze: esplorazione dello spazio delle soluzioni con i modelli del Calcolo Combinatorio (Ricorsione e problem-solving: 3.2, 3.3)
Sia dato un grafo semplice non orientato G = (V, A) di N vertici, identificati da interi nell'intervallo 0..N-1, e E archi, identificati come coppie di vertici. Il grafo è memorizzato su di un file, nella cui prima riga compaiono N ed E, mentre nelle E righe successive compaiono, uno per riga, gli archi nella forma u v.

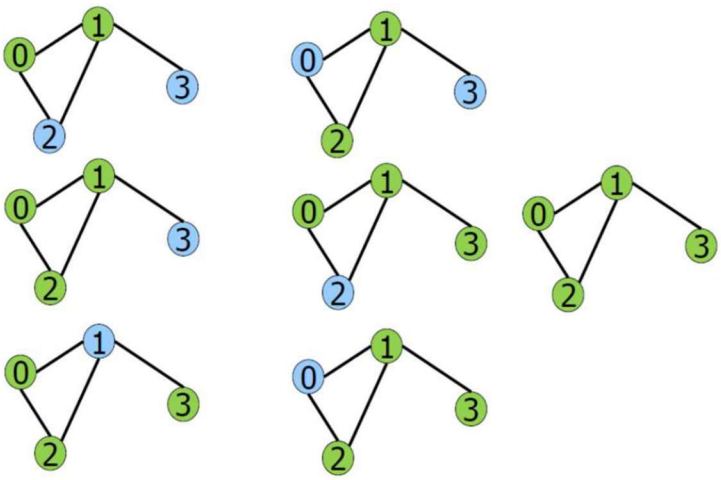
Un vertex cover di G=(V,A) è un sottoinsieme $W \subseteq V$ tale che, per ogni arco $(u,v) \in A$, almeno uno dei suoi estremi appartenga a W: $u \in W$ OR $v \in W$.

Dopo aver letto da file il grafo G ed aver memorizzato le informazioni rilevanti in opportune strutture dati, si elenchino tutti i vertex cover.

Esempio: per il grafo seguente



esistono i seguenti vertex cover: {0,1}, {1,2}, {0,1,2}, {0,1,3}, {0,2,3}, {1,2,3}, {0,1,2,3}.



Si osservi che questo esercizio non richiede conoscenze di Teoria dei Grafi. Si identifichi il modello del Calcolo Combinatorio utilizzato.

Esercizio n.2: **Playlist**
Competenze: esplorazione dello spazio delle soluzioni con i modelli del Calcolo Combinatorio (Ricorsione e problem-solving: 3.2, 3.3)

Un gruppo di `na` amici, con `na` ≥ 1, vuole preparare una playlist per un viaggio. Ogni amico propone da una a cinque canzoni. La playlist deve contenere esattamente `na` canzoni, scegliendo una e una sola canzone tra quelle proposte da ciascun amico. L'ordine dei brani nella playlist coincide con l'ordine degli amici nel file. Tutte le canzoni proposte sono distinte, anche tra amici diversi.

Le informazioni sono memorizzate in un file testuale denominato `brani.txt`, organizzato come segue:

- sulla prima riga compare il numero `na` di amici;
- seguono `na` blocchi, uno per ciascun amico, nell'ordine. Ogni blocco contiene un intero C_i, con $1 \leq C_i \leq 5$, sulla prima riga, seguito dai titoli delle C_i canzoni proposte, uno per riga.

Si assuma che tutti i titoli siano privi di spazi e lunghi al più 255 caratteri.

Si scriva un programma in C che, letto il file di input e memorizzate le informazioni in opportune strutture dati, generi e stampi tutte le playlist possibili secondo le regole indicate.

Nota: il file di esempio allegato è indentato per distinguere più facilmente i blocchi relativi ai diversi amici; gli spazi e le tabulazioni iniziali non fanno parte dei titoli.

Si consiglia di individuare preventivamente il modello del Calcolo Combinatorio, tra quelli visti a lezione, a cui fa riferimento il problema e di adattare il codice presentato in aula.

Esercizio n.3: Collane e pietre preziose
Competenze: esplorazione dello spazio delle soluzioni con i modelli del Calcolo Combinatorio (Ricorsione e problem-solving: 3.2.4, 3.3.6), problemi di ottimizzazione (Ricorsione e problem solving: 3.5)

Un gioielliere ha a disposizione **z zaffiri, r rubini, t topazi e s smeraldi**, con z, r, t e s interi non negativi, e vuole creare una collana disponendo le pietre una dopo l'altra.
La collana è considerata una sequenza aperta: le regole seguenti si applicano a ogni coppia di pietre consecutive, senza imporre alcun vincolo tra l'ultima e la prima pietra:

- uno zaffiro può essere seguito soltanto da uno zaffiro o da un rubino;
- un rubino può essere seguito soltanto da uno smeraldo o da un topazio;
- un topazio può essere seguito soltanto da uno zaffiro o da un rubino;
- uno smeraldo può essere seguito soltanto da uno smeraldo o da un topazio.

Le pietre dello stesso tipo sono indistinguibili. Non è necessario utilizzare tutte le pietre disponibili, ma il numero di pietre utilizzate per ciascun tipo non può superare la relativa disponibilità.

Si scriva una funzione C che **calcoli la lunghezza e visualizzi la composizione di una collana di lunghezza massima** che rispetti i vincoli indicati. La lunghezza della collana è il numero di pietre che la compongono. Se esistono più soluzioni di lunghezza massima, è sufficiente visualizzarne una.

Osservazione: la lunghezza della soluzione non è nota a priori. Se è disponibile almeno una pietra, essa è compresa tra 1 e z+r+t+s; se non sono disponibili pietre, la lunghezza è 0 e la collana è vuota. Si identifichi il modello del Calcolo Combinatorio e si valuti la possibilità di introdurre criteri di pruning.

Nota: si presti attenzione alla crescita del tempo di esecuzione necessario per individuare una soluzione ottima all'aumentare del numero di pietre disponibili.

Esercizio n.4: Collane e pietre preziose (versione 2)

Si consideri il contesto introdotto dall'esercizio precedente, mantenendo le disponibilità delle pietre, la natura di sequenza aperta della collana e tutte le regole di composizione già definite.

Ogni tipologia di pietra è caratterizzata da un valore unitario intero non negativo: val_z per gli zaffiri, val_r per i rubini, val_t per i topazi e val_s per gli smeraldi.

Il valore della collana è dato dalla somma dei valori delle singole pietre che la compongono. Indicando con n_z, n_r, n_t e n_s il numero di zaffiri, rubini, topazi e smeraldi utilizzati nella collana, il suo valore è:

$$val = val_z * n_z + val_r * n_r + val_t * n_t + val_s * n_s$$

Oltre alle regole dell'esercizio precedente, devono essere rispettati i seguenti vincoli:

- ogni sequenza di pietre consecutive dello stesso tipo deve avere lunghezza non superiore a max_rip, dove max_rip è un intero positivo assegnato;
- il numero di zaffiri utilizzati nella collana non può superare il numero di smeraldi utilizzati: $n_z \leq n_s$.

Non è necessario utilizzare tutte le pietre disponibili. È ammessa la collana vuota, di lunghezza e valore pari a 0.

Si scriva una funzione C che calcoli il valore massimo ottenibile e visualizzi la composizione di una collana che lo realizza, rispettando tutti i vincoli indicati. Se esistono più soluzioni di valore massimo, è sufficiente visualizzarne una, indipendentemente dalla sua lunghezza.

Valutazione: Gli esercizi 1, 2 ed uno a scelta tra l'esercizio 3 e 4 saranno oggetto di valutazione
Scadenza caricamento di quanto valutato: entro le 23:59 del 23/11/2026.